Parallel Computing

A/Prof. Dr. Tran Van Hoai, Dr. Diep Thanh Dang

Assignment 2

(October 23, 2026)

1 The A* Algorithm in the Sequential World

In the lecture, we discussed a sequential version of the A* algorithm. This usually helps us to find the (near-)optimal path to get a destination from a source location. In this assignment, we are about to do the following:

- Implementing a sequential program using the sequential A* algorithm to find the (near-)optimal path from two arbitrary locations in a certain real-life road network (e.g., Ho Chi Minh City).
- Considering the following circumstances: the h function is dynamic, uncertain; the f function is a vector function. Re-implementing the sequential A* algorithm under the circumstances.
- Devising the approach to testing the correctness of your implementations.

2 The Concurrent Set Data Structure

In the lecture, we discussed a concurrent version of the set data structure represented as a sorted linked list. This concurrent list has several variants in terms of synchronization strategies, including course-grained locking (acquiring a lock on every method call), fine-grained locking (acquiring locks for individual item nodes), optimistic schemes (traversing the list without locks, then locking and validating), and lazy schemes (deferring unlinking item nodes). In this assignment, we need to conduct the following:

- Implementing any two of the list variants. You can use your own locks from the previous assignment, or the built-in ones offered by the used programming languages.
- Devising the approach to testing the correctness of your implementations.
- Evaluating and comparing the performance of your list implementations based on a usage scenario of your choice. Note that the exponential backoff (mentioned in the previous assignment) can be used in some concurrent data structures to minimize contention between threads.

3 The A* Algorithm in the Parallel World

In order to realize a parallel version of the sequential A* algorithm, a concurrent data structure with priority is usually utilized. The sorted list (described in Section 2) is such a data structure. However, it may not work if you combine the sequential A* algorithm (elaborated in Section 1) with the sorted list without any modification. In this assignment, you are supposed to complete the following:

- Adapting both the sequential A* algorithm and the concurrent sorted list so that they can be merged into a parallel A* algorithm version.
- Devising the approach to testing the correctness of the parallel A* algorithm.
- Conducting a performance and scalability study of all the possible sequential and parallel A* implementation versions on all shared-memory platforms you possess. Considering the quality of outcomes and the execution time as performance metrics. Note that the exponential backoff (mentioned in the previous assignment) can be used in some concurrent data structures to minimize contention between threads.

4 Other Optimizations (optional)

In addition to the concurrent data structure with priority, several other optimization techniques, e.g. contraction hierarchies, pruning, labeling, and so on, can be applied to the sequential or parallel A* algorithms. Such techniques are encouraged to be employed in your implementations as well.

Other Requirements

This assignment must be conducted in groups, each of which consists of either 2 or 3 members. You must submit a zip file containing your final pdf report as well as all relevant Python/C++ or mixed source code to our course website by December 5, 2026. Especially, the number of experimented shared-memory platforms must be at least be equal to the member count in your group.